

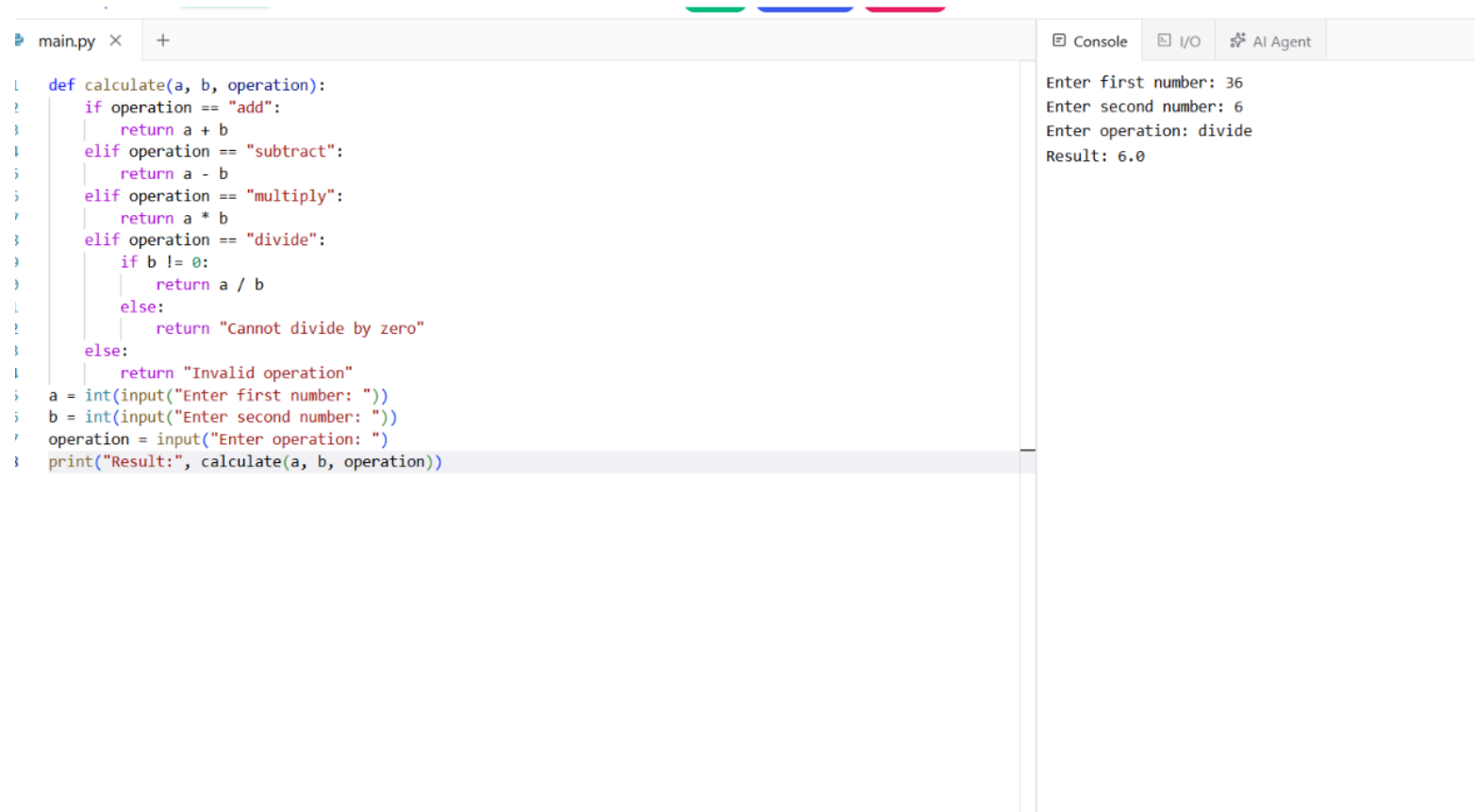
1. Write a function `calculate(a, b, operation)` that performs addition, subtraction, multiplication, or division based on the supplied operation.

Program

```
def calculate(a, b, operation):
    if operation == "add":
        return a + b
    elif operation == "subtract":
        return a - b
    elif operation == "multiply":
        return a * b
    elif operation == "divide":
        if b != 0:
            return a / b
        else:
            return "Cannot divide by zero"
    else:
        return "Invalid operation"

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
operation = input("Enter operation: ")
print("Result:", calculate(a, b, operation))
```

Output

The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'calculate' that takes three arguments: 'a', 'b', and 'operation'. It uses if-elif-else statements to perform addition, subtraction, multiplication, and division. For division, it checks if 'b' is not zero to avoid a division by zero error. If the operation is not one of the specified ones, it returns 'Invalid operation'. Below the function definition, the code prompts the user for the first number, second number, and operation, then prints the result. The output console on the right shows the user input: '36' for the first number, '6' for the second number, and 'divide' for the operation. The final output is 'Result: 6.0'.

```
def calculate(a, b, operation):
    if operation == "add":
        return a + b
    elif operation == "subtract":
        return a - b
    elif operation == "multiply":
        return a * b
    elif operation == "divide":
        if b != 0:
            return a / b
        else:
            return "Cannot divide by zero"
    else:
        return "Invalid operation"

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
operation = input("Enter operation: ")
print("Result:", calculate(a, b, operation))
```

Console

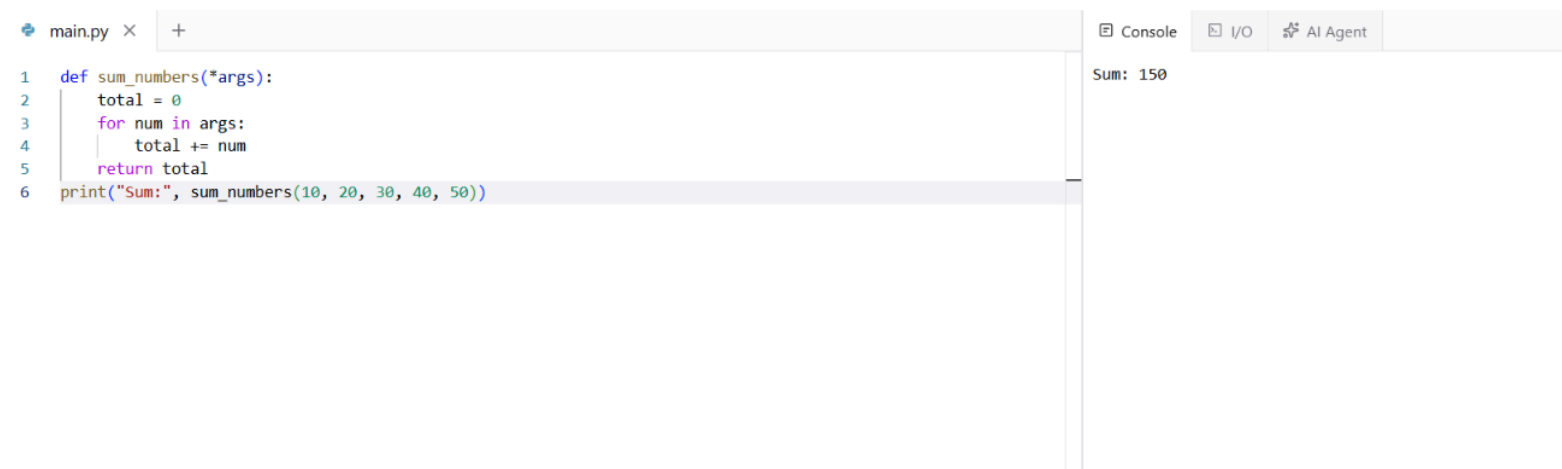
Enter first number: 36
Enter second number: 6
Enter operation: divide
Result: 6.0

2. Write a function `sum_numbers(*args)` that accepts any number of arguments and returns their sum.

Program

```
def sum_numbers(*args):
    total = 0
    for num in args:
        total += num
    return total
print("Sum:", sum_numbers(10, 20, 30, 40, 50))
```

Output

A screenshot of a code editor interface. The top bar shows a file named 'main.py' with a close button and a plus sign for additional files. Below the bar, the code editor displays the following Python code:

```
1 def sum_numbers(*args):
2     total = 0
3     for num in args:
4         total += num
5     return total
6 print("Sum:", sum_numbers(10, 20, 30, 40, 50))
```

The code is syntax-highlighted. To the right of the code editor is a console panel with tabs for 'Console', 'I/O', and 'AI Agent'. The 'Console' tab is active, showing the output: 'Sum: 150'.

3. Write a function `employee(**args)` that accepts employee information such as name, ID, department and salary, then displays the information.

Program

```
def employee(**args):
    print("Employee Information:")
    for key, value in args.items():
        print(key, ":", value)
employee(
    name="Indu",
    ID=101,
    department="AI&DS",
    salary=35000
)
```

Output

main.py × +

```
1 def employee(**args):
2     print("Employee Information:")
3     for key, value in args.items():
4         print(key, ":", value)
5 employee(
6     name="Indu",
7     ID=101,
8     department="AI&DS",
9     salary=35000
10 )
```

Console I/O AI Agent

Employee Information:
name : Indu
ID : 101
department : AI&DS
salary : 35000

4. Write a function `remove_duplicates(lst)` that returns a list containing only unique elements while preserving their original order.

Program

```
def remove_duplicates(lst):
    unique = []
    for item in lst:
        if item not in unique:
            unique.append(item)
    return unique

numbers = [10, 20, 10, 30, 20, 40, 30, 50]
print("Original list:", numbers)
print("List after removing duplicates:", remove_duplicates(numbers))
```

Output

main.py × +

```
1 def remove_duplicates(lst):
2     unique = []
3     for item in lst:
4         if item not in unique:
5             unique.append(item)
6     return unique
7 numbers = [10, 20, 10, 30, 20, 40, 30, 50]
8 print("Original list:", numbers)
9 print("List after removing duplicates:", remove_duplicates(numbers))
```

Console I/O AI Agent

Original list: [10, 20, 10, 30, 20, 40, 30, 50]
List after removing duplicates: [10, 20, 30, 40, 50]

5.Using a lambda function, sort a list of tuples based on the second element. Example: [(1,5), (2,3), (4,1)].

Program

```
numbers = [(1, 5), (2, 3), (4, 1)]
sorted_list = sorted(numbers, key=lambda x: x[1])
print("Original list:", numbers)
print("Sorted list:", sorted_list)
```

Output

main.py × +

```
1 numbers = [(1, 5), (2, 3), (4, 1)]
2 sorted_list = sorted(numbers, key=lambda x: x[1])
3 print("Original list:", numbers)
4 print("Sorted list:", sorted_list)
```

Console I/O AI Agent 128

Original list: [(1, 5), (2, 3), (4, 1)]
Sorted list: [(4, 1), (2, 3), (1, 5)]